

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Total Marks: 7.5

Obtained Marks: _____

OOP LAB

Final Project

Last Date of Submission: 18-05-2026

Submitted To: Sir Hassan Mujtaba

Student Name: Aswad Nasir, Syed Rohail

Registration Number: 23108337 , 23108387

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

1. Introduction:

The **Bank Management System** is a software application developed to manage basic banking operations digitally. This project is designed using **Object-Oriented Programming (OOP)** concepts in **C++**.

The system helps users perform banking tasks such as:

- Creating bank accounts
- Depositing money
- Withdrawing money
- Viewing account details
- Adding interest to savings accounts

The project reduces manual work and demonstrates how OOP principles can be applied in real-world applications.

2. Problem Statement:

Traditional manual banking systems are time-consuming and difficult to manage when the number of customers increases. Maintaining records manually may also lead to errors and data loss.

This project aims to develop a computerized banking system that efficiently manages customer accounts and transactions using OOP concepts.

3. Objectives:

The main objectives of this project are:

- To implement a digital banking system
- To apply OOP concepts practically
- To manage customer account records efficiently
- To provide secure deposit and withdrawal operations
- To reduce human effort and calculation errors

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

4. Scope of the Project:

The system can be used in small banking environments for educational purposes. It supports:

- Account creation
- Balance management
- Interest calculation
- Transaction handling
- Displaying customer details

Future enhancements may include:

- Database integration
- ATM simulation
- Online banking
- Password protection
- File handling
- GUI interface

5. OOP Concepts Used:

OOP Concept	Description
Classes & Objects	Used to represent bank accounts
Encapsulation	Data members kept protected/private
Inheritance	SavingsAccount inherits BankAccount
Polymorphism	display() function overridden
Abstraction	Complex operations hidden through functions

6. System Design:

Main Classes

1. BankAccount

Base class containing:

- Account number
- Holder name
- Balance
- Deposit function
- Withdraw function

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

2. SavingsAccount

Derived class containing:

- Interest rate
- Interest calculation feature

7. Functional Requirements:

The system should be able to:

- Create new accounts
- Store account information
- Deposit money
- Withdraw money
- Show account details
- Add interest to savings accounts

8. Non-Functional Requirements:

- User-friendly interface
- Fast processing
- Easy maintenance
- Secure operations
- Reliable performance

9. Advantages of the System:

- Reduces paperwork
- Faster calculations
- Easy account management
- Improved accuracy
- Better record organization

10. Conclusion:

The **Bank Management System** is an efficient OOP-based application that demonstrates important programming concepts while solving real-world banking problems. The project improves understanding of object-oriented design and software development techniques.

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Code:

```
#include <iostream>
using namespace std;

class BankAccount {
protected:
    int accountNumber;
    string holderName;
    double balance;

public:
    BankAccount() {
        accountNumber = 0;
        holderName = "";
        balance = 0;
    }

    void createAccount(int accNo, string name, double bal) {
        accountNumber = accNo;
        holderName = name;
        balance = bal;
    }

    void deposit(double amount) {
        balance += amount;
        cout << "Amount Deposited Successfully!\n";
    }

    void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
            cout << "Withdrawal Successful!\n";
        } else {
            cout << "Insufficient Balance!\n";
        }
    }

    virtual void display() {
        cout << "\nAccount Number : " << accountNumber << endl;
        cout << "Account Holder : " << holderName << endl;
        cout << "Balance      : " << balance << endl;
    }

    int getAccountNumber() {
        return accountNumber;
    }
};
```

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

```
class SavingsAccount : public BankAccount {
private:
    double interestRate;

public:
    void setInterestRate(double rate) {
        interestRate = rate;
    }

    void addInterest() {
        balance += balance * interestRate / 100;
        cout << "Interest Added Successfully!\n";
    }

    void display() override {
        BankAccount::display();
        cout << "Interest Rate : " << interestRate << "%\n";
    }
};

int main() {

    SavingsAccount accounts[100]; // Array for storing accounts
    int totalAccounts = 0;

    int choice;

    do {
        cout << "\n===== BANK MANAGEMENT SYSTEM =====\n";
        cout << "1. Create Account\n";
        cout << "2. Deposit Money\n";
        cout << "3. Withdraw Money\n";
        cout << "4. Display Accounts\n";
        cout << "5. Add Interest\n";
        cout << "0. Exit\n";

        cout << "Enter Choice: ";
        cin >> choice;

        if (choice == 1) {

            int accNo;
            string name;
            double balance, rate;

            cout << "Enter Account Number: ";
```

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

```
cin >> accNo;

cin.ignore();

cout << "Enter Holder Name: ";
getline(cin, name);

cout << "Enter Initial Balance: ";
cin >> balance;

cout << "Enter Interest Rate: ";
cin >> rate;

accounts[totalAccounts].createAccount(accNo, name, balance);
accounts[totalAccounts].setInterestRate(rate);

totalAccounts++;

cout << "Account Created Successfully!\n";
}

else if (choice == 2) {

    int accNo;
    double amount;

    cout << "Enter Account Number: ";
    cin >> accNo;

    cout << "Enter Amount: ";
    cin >> amount;

    for (int i = 0; i < totalAccounts; i++) {
        if (accounts[i].getAccountNumber() == accNo) {
            accounts[i].deposit(amount);
        }
    }
}

else if (choice == 3) {

    int accNo;
    double amount;

    cout << "Enter Account Number: ";
    cin >> accNo;

    cout << "Enter Amount: ";
```

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

```
cin >> amount;

for (int i = 0; i < totalAccounts; i++) {
    if (accounts[i].getAccountNumber() == accNo) {
        accounts[i].withdraw(amount);
    }
}

else if (choice == 4) {

    for (int i = 0; i < totalAccounts; i++) {
        accounts[i].display();
    }
}

else if (choice == 5) {

    int accNo;

    cout << "Enter Account Number: ";
    cin >> accNo;

    for (int i = 0; i < totalAccounts; i++) {
        if (accounts[i].getAccountNumber() == accNo) {
            accounts[i].addInterest();
        }
    }
}

} while (choice != 0);

cout << "\nThank You for Using Bank Management System!\n";

return 0;
}
```

Output:

```
C:\Users\23108387\Desktop\l  X  +  v

===== BANK MANAGEMENT SYSTEM =====
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Display All Accounts
5. Add Interest
0. Exit
Enter Choice: 1
Enter Account Number: 134141
Enter Holder Name: rohail
Enter Initial Balance: 2000000000
Enter Interest Rate: 2
Account Created Successfully!

===== BANK MANAGEMENT SYSTEM =====
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Display All Accounts
5. Add Interest
0. Exit
Enter Choice: A|
```